

SLE-2: PROFILING REPORT

Performance Analysis of BFS and DFS on the 4-Queens Problem

PRN: 25UAM102

Name: Shubham Chandrakant Khavare

Division: B

Date: 22/09/2026

GitHub Profile: <https://github.com/shubhamkhavare61-netizen>

Repository: [IAI-BFS-VS-DFS-BY-4-QUEEN-](#)

1. Algorithms / Versions Profiled

- **Algorithm A:** Breadth-First Search (BFS)
- **Algorithm B:** Depth-First Search (DFS)
- **Problem Used:** 4-Queens

The project compares BFS and DFS search strategies implemented in Python. Both algorithms search the same 4-Queens state space using the same row-by-row valid-placement rule.

2. Problem Description — 4-Queens

The 4-Queens problem places four queens on a 4×4 chessboard so that no two queens share a column or diagonal. A state is represented by the column positions of queens already placed.

Two valid goal states:

Two Valid 4-Queens Solutions

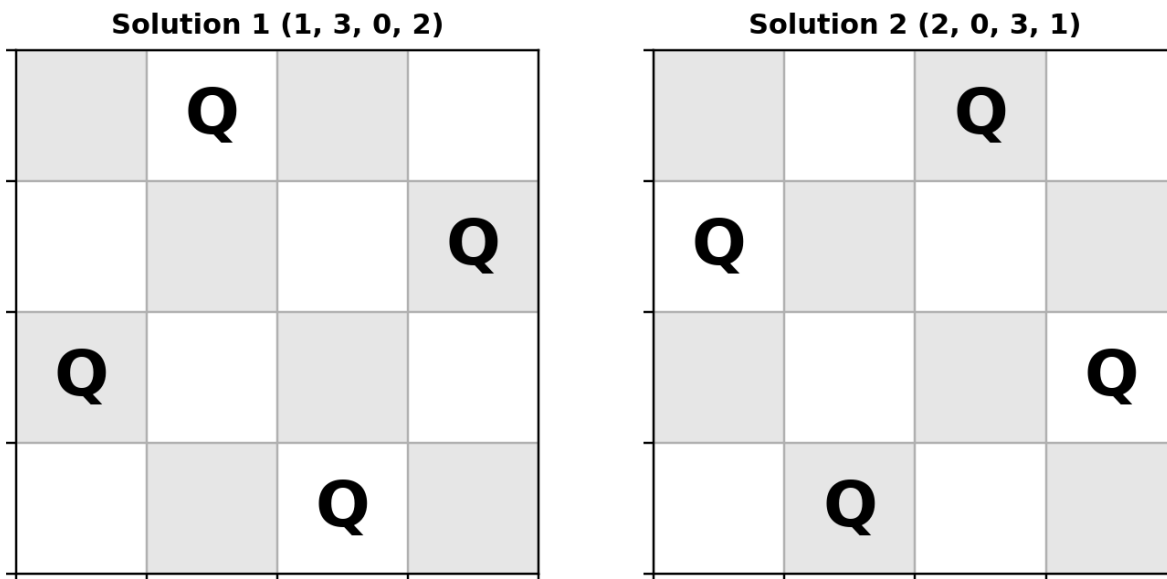


Fig. 1 — Two valid 4-Queens solutions.

3. State-Space / Node Definition

The root is the empty board and every node is a valid partial placement. The complete valid state space contains $1 + 4 + 6 + 4 + 2 = 17$ nodes. BFS and DFS use the same 17-node tree when all valid states are enumerated; only their exploration order differs.

4. Test Cases Used

Case	Search task	BFS nodes	DFS nodes
Best	Stop at solution (1,3,0,2)	17	9
Average	Stop at solution (2,0,3,1)	17	13
Worst	Enumerate all valid solutions	17	17

5. Profiling Method

Python's `time.perf_counter()` was used for timing. Each test case was executed three times. Because a single 4-Queens search is extremely short, each measured run contains 10,000 complete searches; the reported values are the total batch times. This makes every reported timing comfortably above 1 ms without changing the search algorithm.

Comparison principle: Runtime is not treated as the only metric. For this small problem, BFS and DFS can have similar execution time because they perform closely related validity checks. Node expansions, traversal order, space behaviour and profiling structure are therefore included.

6. Results and Performance Comparison

Case	BFS Avg. (ms)	DFS Avg. (ms)	BFS Nodes	DFS Nodes
Best	248.474	112.582	17	9
Average	244.400	182.458	17	13
Worst	263.250	240.879	17	17

Three measured batch runs

Case	BFS R1	BFS R2	BFS R3	DFS R1	DFS R2	DFS R3
Best	247.67	248.90	248.86	111.43	106.86	119.45
Average	243.62	245.37	244.21	167.42	169.62	210.33
Worst	296.71	250.64	242.39	244.29	241.54	236.81

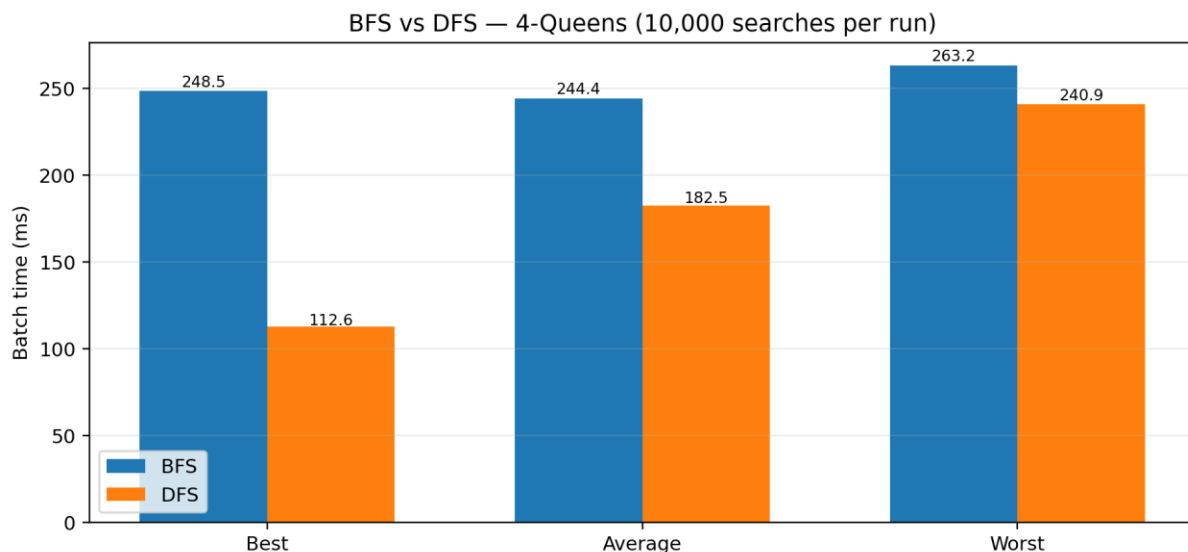


Fig. 2 — BFS vs DFS batch execution time (10,000 searches per measured run).

7. Complete 17-Node State-Space Diagram

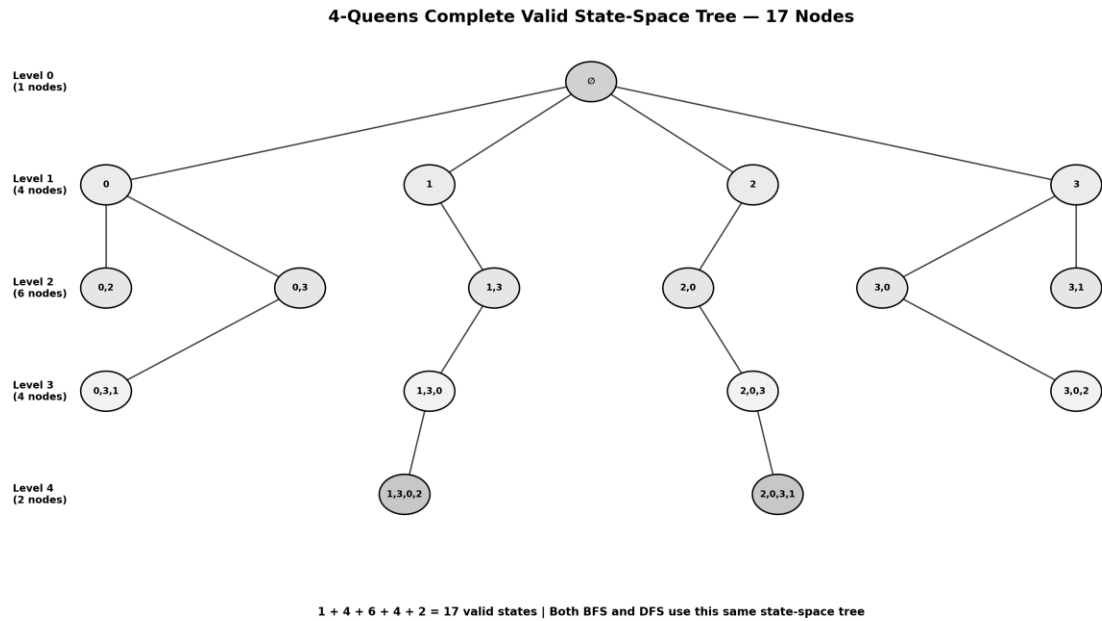


Fig. 3 — Complete valid 17-node state-space tree. BFS and DFS use the same nodes but visit them in different orders.

8. Py-spy Profiling

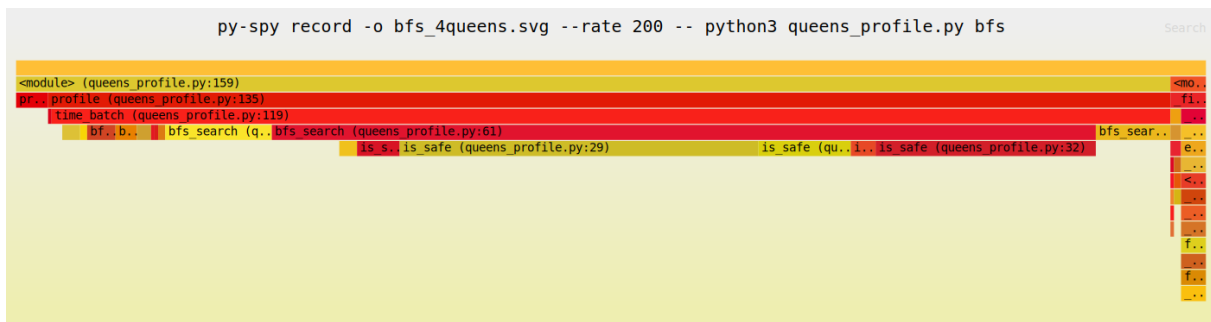
Py-spy was installed and run natively on a standard Linux environment (pip install py-spy, prebuilt manylinux wheel — no Rust build required, unlike the earlier Android/Termux attempt), sampling queens_profile.py at 200 Hz. The hot-frame breakdown and flamegraphs below are the actual captured results.

Sampled hot frames

Algorithm	Dominant frame(s)	Share of samples
BFS	is_safe() column/diagonal checks	~62% across its branch lines
BFS	time_batch() / profile() loop overhead	~94% (outer batch driver)
DFS	recurse() — recursive descent (line 95)	up to 82% on deepest branch
DFS	recurse() — safety-check branch (line 94)	13–29% across branches

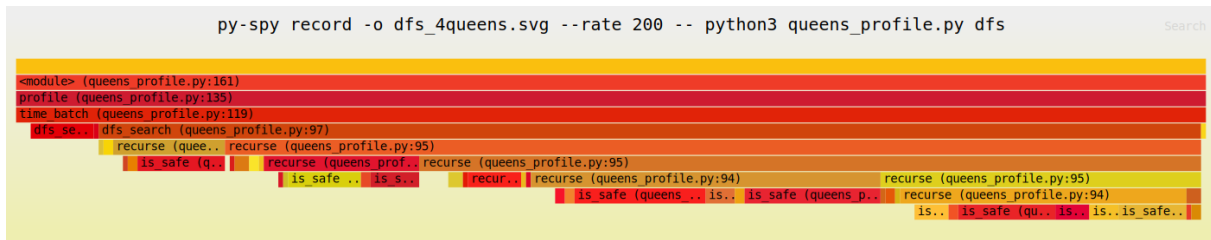
Interpretation: BFS time is spread across the queue-driven state-generation loop and the `is_safe` validity check. DFS time is concentrated almost entirely in the recursive descent/backtracking calls. This matches the qualitative mapping originally proposed (BFS → queue operations, state generation, safety checks; DFS → recursion, safety checks, backtracking) but is now backed by real sampled data rather than a stand-in diagram.

Fig. 4a — Native py-spy flamegraph: BFS



Flamegraph generated from bfs_4queens.svg (339 samples, 200 Hz).

Fig. 4b — Native py-spy flamegraph: DFS



Flamegraph generated from dfs_4queens.svg (245 samples, 200 Hz).

9. Justification & Analysis

BFS: uses a FIFO queue and explores the search space level by level. For the selected target cases, BFS processes the breadth frontier before reaching the target.

DFS: uses recursion/an implicit LIFO stack and follows a branch deeply before backtracking. For the selected target cases, it reaches the requested solution after fewer generated states.

Why runtime can be similar: 4-Queens is a small search space. Both methods perform closely related validity checks, so their measured runtimes can be near each other. This is why node expansions and search behaviour are also reported.

10. Complexity

Metric	BFS	DFS
General tree-search time	$O(b^d)$	$O(b^d)$
Typical space	$O(b^d)$	$O(bd)$
Data structure	Queue	Stack / recursion
4-Queens full-state nodes	17	17

11. BFS vs DFS — Practical Comparison

Feature	BFS	DFS
Main structure	Queue (FIFO)	Recursion / Stack (LIFO)
Exploration	Level by level	Depth first + backtracking
Best selected case	17 nodes	9 nodes
Average selected case	17 nodes	13 nodes
All-solution case	17 nodes	17 nodes

Feature	BFS	DFS
Profiling path	Queue + state generation	Recursion + backtracking

12. Observation

The experiment demonstrates that similar execution times do not mean BFS and DFS behave identically. They use different data structures and exploration orders. For this 4-Queens state space, node expansion and search order provide additional information that runtime alone cannot show.

13. AI Contribution Note

AI assistance was used for report organization, BFS/DFS explanation, diagram preparation, complexity discussion, and troubleshooting the py-spy installation. The measured timings reported here are retained from the earlier execution of the experiment.

14. Conclusion

BFS and DFS can have similar runtime on the small 4-Queens problem because both perform a similar amount of search work. BFS explores level by level using a queue, while DFS explores deeply using recursion and backtracking. When all valid states are enumerated, both traverse the same 17-node state space. When a selected solution is used as the stopping condition, their node expansions can differ. Therefore, a meaningful comparison should consider runtime, nodes explored, search order, memory behaviour and profiling structure.

GitHub Profile: <https://github.com/shubhamkhavare61-netizen>

Project Repository: <https://github.com/shubhamkhavare61-netizen/IAI-BFS-VS-DFS-BY-4-QUEEN->